

Taller: Structs, Mapas y Manejo de Archivos en Elixir

Objetivo

Aplicar los siguientes conceptos:

- Structs
- Mapas (Map)
- Manejo de archivos (CSV y JSON)
- Manejo de errores

Entrega

Se debe crear un repositorio en GitHub que incluya:

- Proyecto creado con Mix usando Application (como se explica en la [Guía de clase](#))
 - Código organizado en módulos
 - Archivo README.md que incluya una breve descripción de la solución, los aprendizajes obtenidos durante el desarrollo del ejercicio y una explicación de cómo se utilizó la inteligencia artificial como apoyo en la solución.
 - Se debe evidenciar la participación de todos los integrantes del grupo mediante commits en el repositorio.
 - Código ejecutable desde consola
-

Ejercicio 1: Gestión de Socios de un Gimnasio

Descripción (Este problema está parcialmente resuelto en la [Guía de clase #8](#)).

Un gimnasio desea gestionar sus socios y las clases a las que están inscritos.

Cada socio tiene:

- Cédula (obligatorio)
- Nombre (obligatorio)
- Edad (obligatorio)
- Lista de clases

Se debe usar un Map donde:

- Clave: cédula
- Valor: struct Socio

Estructura requerida

Se deben implementar los siguientes módulos:

- Socio: definición del struct y validaciones

- Gimnasio: lógica de negocio
- GestionArchivos: lectura y escritura en CSV
- Menu: interacción por consola. Todas las funcionalidades deben poder hacerse desde consola.

Funcionalidades

El sistema debe permitir:

1. Crear un socio
2. Eliminar un socio
3. Inscribir a un socio en una clase
4. Desinscribir a un socio de una clase
5. Buscar un socio por cédula
6. Listar todos los socios
7. Listar todos los socios que estén inscritos en una clase específica
8. Listar todas las clases de un socio dada su cédula

Persistencia

Archivo: `socios.csv`

Formato:

```
cedula,nombre,edad,clases
123,Juan Pérez,30,Yoga;Pilates
456,Ana García,25,CrossFit;Yoga
```

Reglas de persistencia

- Cargar datos al iniciar el programa
- Guardar automáticamente después de cada operación
- Crear el archivo si no existe
- Manejar errores de lectura y escritura

Validaciones

- No permitir cédulas duplicadas
- La edad debe ser un número positivo
- No permitir clases duplicadas por socio

Manejo de errores

Todas las funciones deben retornar:

```
{:ok, resultado}
{:error, motivo}
```

Ejercicio 2: Sistema de Inventario

Descripción (Este problema está definido en la [Guía de clase #8](#)).

Implementar un sistema de inventario de productos con énfasis en consultas y procesamiento funcional.

Cada producto tiene:

- Código (obligatorio)
- Nombre (obligatorio)
- Precio (obligatorio)
- Cantidad (obligatorio)

Se debe usar un Map donde:

- Clave: código
- Valor: struct Producto

Estructura requerida

Módulos:

- Producto: struct y validaciones
- Inventario: lógica de negocio
- ArchivoJSON: persistencia
- Menu: interacción por consola

Funcionalidades

- Agregar producto
- Actualizar producto
- Eliminar producto
- Listar productos

Validaciones obligatorias

- No permitir códigos repetidos
- El código debe tener máximo 5 caracteres
- El nombre solo puede contener letras
- El precio debe ser mayor o igual a 0
- La cantidad debe ser un número entero mayor o igual a 0

Persistencia

Archivo: `productos.json`. Se debe usar la librería **Jason** para escribir y leer.

Reglas de persistencia

- Cargar datos al iniciar

- Guardar automáticamente después de cada operación
- Manejar errores de lectura y escritura

Consultas

Implementar las siguientes funciones utilizando Enum:

1. Productos cuyo nombre contenga al menos dos vocales. Retornar una lista de tuplas: [{codigo, nombre}]
2. Productos cuyo nombre comience y termine con la misma letra
3. Productos con precio por debajo de un valor dado
4. Los tres productos más caros del inventario
5. Productos con precio entre dos valores Retornar una cadena: "Producto1 - 50000, Producto2 - 70000"
6. Agrupar productos por rango de precio:
 - Menores a 50000
 - Entre 50000 y 100000
 - Mayores a 100000

Pruebas

Se deben implementar al menos dos pruebas usando ExUnit.

Menú por consola

Debe existir un menú que permita acceder a todas las funcionalidades.

Ejemplo:

1. Crear socio
2. Listar socios
3. Inscribir a clase
- ...
0. Salir

Buenas prácticas obligatorias

- Código modular
- Manejo de errores mediante tuplas
- Funciones pequeñas y claras